# Understanding autoregressive models from a mathematical view

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

November 20, 2025

# Feynman's words

What I cannot create, I do not understand.

# Outline

# What are autoregressive models?

Given a sequence $\mathbf{x}_{1:T} = (x_1, x_2, \ldots, x_T)$, autoregressive models factorize the joint probability as:

$$p(\mathbf{x}_{1:T}) = \prod_{t=1}^{T} p(x_t | \mathbf{x}_{t-1:1}) \tag{1}$$

where $\mathbf{x}_{t-1:1} = (x_{t-1}, x_{t-2}, \ldots, x_1)$ represents the history.
Key properties:

- Sequential generation: Generate one token at a time
- Conditional independence: Each token depends only on previous tokens
- Causal structure: No future information leakage

# Mathematical foundation: Chain rule of probability

The autoregressive factorization follows directly from the chain rule of probability:

$$p(\mathbf{x}_{1:T}) = p(x_1) \prod_{t=2}^{T} p(x_t|\mathbf{x}_{t-1:1}) \tag{2}$$

This factorization is:

- **Always valid**: No assumptions required
- **Exact**: No approximation involved
- **Order-dependent**: Different orderings give different factorizations

The challenge is modeling the conditional distributions $p(x_t|\mathbf{x}_{t-1:1})$.

# Why autoregressive models?

Advantages of autoregressive modeling:

1. **Simplicity**: Direct application of maximum likelihood
2. **Interpretability**: Clear probabilistic interpretation
3. **Flexibility**: Can model complex dependencies
4. **Training stability**: Well-understood optimization landscape

Challenges:

1. **Sequential generation**: Cannot parallelize during inference
2. **Error propagation**: Mistakes compound over time
3. **Long-range dependencies**: Hard to capture distant relationships

# Outline

# Maximum Likelihood Estimation

Given a dataset $\mathcal{D} = \{\mathbf{x}_{1:T}^{(i)}\}_{i=1}^{N}$, we maximize the log-likelihood:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \log p(\mathbf{x}_{1:T}^{(i)}; \boldsymbol{\theta}) \tag{3}$$

Substituting the autoregressive factorization:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \sum_{t=1}^{T} \log p(x_t^{(i)} | \mathbf{x}_{t-1:1}^{(i)}; \boldsymbol{\theta}) \tag{4}$$

This decomposes into independent optimization problems for each conditional distribution.

# Cross-entropy loss and its interpretation

For discrete sequences, the loss becomes:

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{N}\sum_{t=1}^{T} \log p(x_t^{(i)} | \mathbf{x}_{t-1:1}^{(i)}; \boldsymbol{\theta}) \tag{5}$$

This is equivalent to minimizing the cross-entropy:

$$H(p_{\text{data}}, p_{\boldsymbol{\theta}}) = \mathbb{E}_{\mathbf{x}_{1:T} \sim p_{\text{data}}} \left[ -\sum_{t=1}^{T} \log p(x_t | \mathbf{x}_{t-1:1}; \boldsymbol{\theta}) \right] \tag{6}$$

**Key insight**: We're learning to predict the next token given the history, which is exactly what we need for generation.

# Teacher forcing and training efficiency

During training, we use **teacher forcing**:

- Input: Ground truth history $\mathbf{x}_{t-1:1}$
- Target: Next token $x_t$
- Loss: Cross-entropy between predicted and true distributions

This allows:

1. **Parallel training**: All time steps computed simultaneously
2. **Stable gradients**: No error propagation during training
3. **Efficient optimization**: Standard backpropagation

Note: Training and inference use different procedures!

# Outline

# Outline

# Recurrent Neural Networks (RNNs)

The simplest autoregressive architecture:

$$h_t = f(h_{t-1}, x_{t-1}; \boldsymbol{\theta}) \tag{7}$$

$$p(x_t | \mathbf{x}_{t-1:1}) = g(h_t; \boldsymbol{\theta}) \tag{8}$$

where $h_t$ is the hidden state and $f, g$ are neural networks.

**Advantages**:

- Natural sequential processing
- Parameter sharing across time
- Variable-length sequences

**Limitations**:

- Vanishing/exploding gradients
- Limited long-range dependencies
- Sequential computation

# Outline

# Long Short-Term Memory (LSTM)

LSTM addresses RNN limitations with gating mechanisms:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(forget gate)} \tag{9}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(input gate)} \tag{10}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \text{(candidate values)} \tag{11}$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad \text{(cell state)} \tag{12}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(output gate)} \tag{13}$$

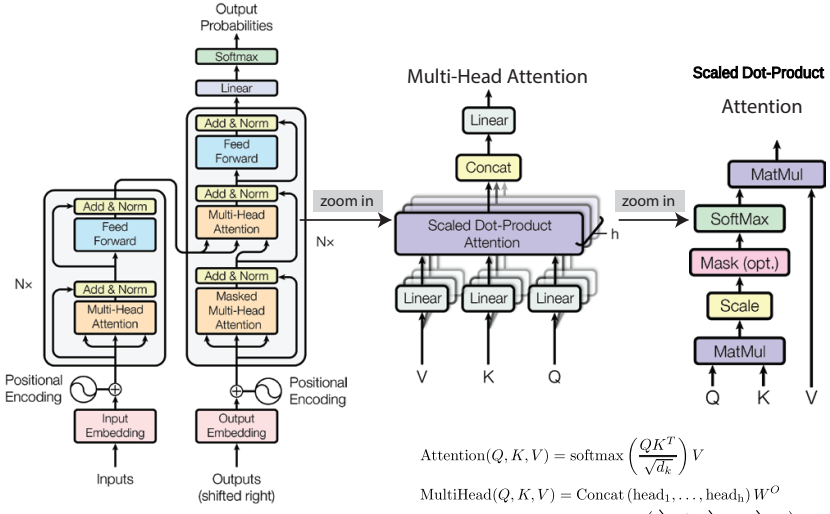
$$h_t = o_t * \tanh(C_t) \quad \text{(hidden state)} \tag{14}$$

**Key innovations**:

- Gating: Control information flow
- Cell state: Long-term memory storage
- Gradient flow: Better backpropagation

# Outline

# Transformers




$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_\text{h}\right)W^O$$

$$\text{where head}_i = \text{Attention}\left(\underbrace{QW_i^Q}_{\text{XW}_i^\text{Q}}, \underbrace{KW_i^K}_{\text{XW}_i^\text{K}}, \underbrace{VW_i^V}_{\text{XW}_i^\text{V}}\right)$$

# Scaled Dot-Product Attention: Key formula

Inputs: queries $Q \in \mathbb{R}^{n \times d_k}$, keys $K \in \mathbb{R}^{n \times d_k}$, values $V \in \mathbb{R}^{n \times d_v}$.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V, \tag{15}$$

where $M$ is an optional mask (e.g., $M_{ij} = -\infty$ for $j > i$ in autoregressive decoding).

- Scaling by $\sqrt{d_k}$ keeps gradients in a reasonable range.
- **Interpretation**: each token re-weights all values using similarity to keys.

# Multi-Head Attention (MHA)

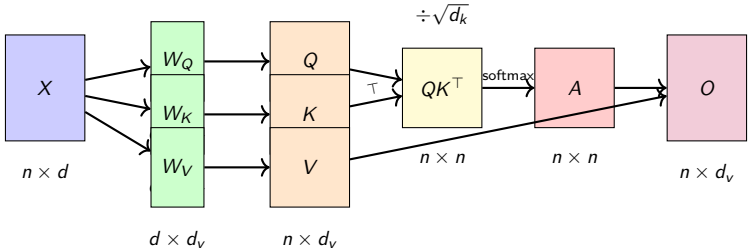
$$\mathrm{MHA}(X) = \mathrm{Concat}(H_1, \ldots, H_h)\, W^O, \tag{16}$$

$$H_i = \mathrm{Attention}(XW_i^Q,\, XW_i^K,\, XW_i^V), \quad i = 1, \ldots, h. \tag{17}$$

- Heads attend to different subspaces/patterns.
- Residual + LayerNorm around MHA and FFN blocks stabilize training.

# Understanding the Transformer architecture from a mathematical view

- Input embedding: $X \in \mathbb{R}^{n \times d}$;
- Query, key, value projections: $Q = XW_Q$, $K = XW_K$, $V = XW_V$; (Notatiion slightly different from the original paper)
- Attention: $A = \mathrm{softmax}(QK^\top / \sqrt{d_k}) + M$;
- Output: $O = AV$;




**Remark**

*Understand the Transformer architecture by block matrix, e.g.,* $(AB)_k = \sum_k A_{ki} B_i$.

# Left Multiplication as Row-wise Linear Combination

Let

$$A \in \mathbb{R}^{m\times n}, \quad B \in \mathbb{R}^{n\times p}, \quad C = AB \in \mathbb{R}^{m\times p}.$$

Denote:

$$A_{i*} = (a_{i1}, \ldots, a_{in}) \quad (\text{row } i \text{ of } A), \qquad B_{k*} \quad (\text{row } k \text{ of } B).$$

By the definition of matrix multiplication,

$$C_{ij} = (AB)_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Therefore the entire $i$-th row of $C$ satisfies

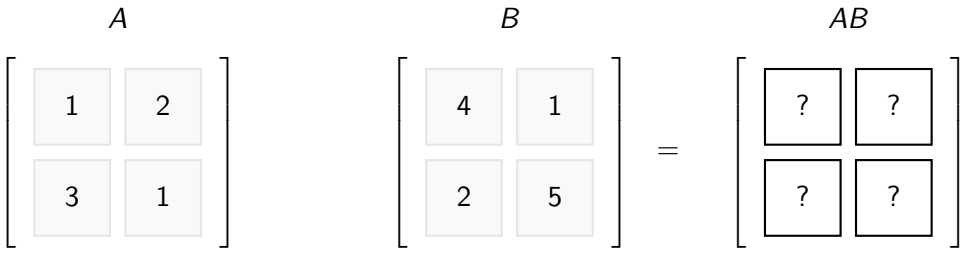
$$C_{i*} = (C_{i1}, \ldots, C_{ip}) = \sum_{k=1}^{n} a_{ik}\, B_{k*}.$$

**Key fact:**

Left multiplying by $A$ takes the rows of $B$ and forms row-wise linear combinations.

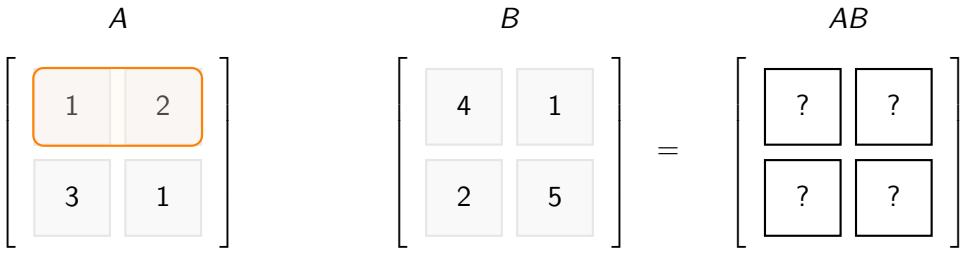
Each row $C_{i*}$ is a linear combination of the rows $B_{k*}$ with coefficients from $A_{i*}$.

# Matrix Multiplication: Row Picture



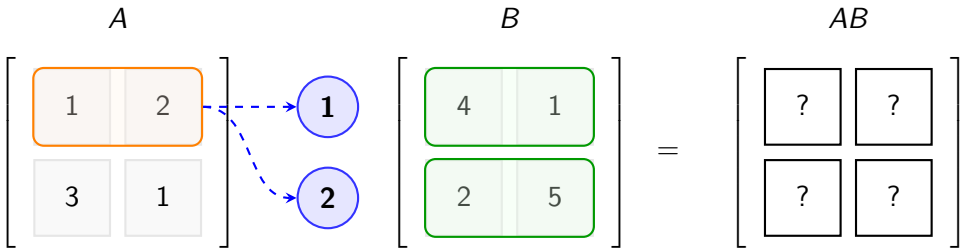

- Initial state

# Matrix Multiplication: Row Picture



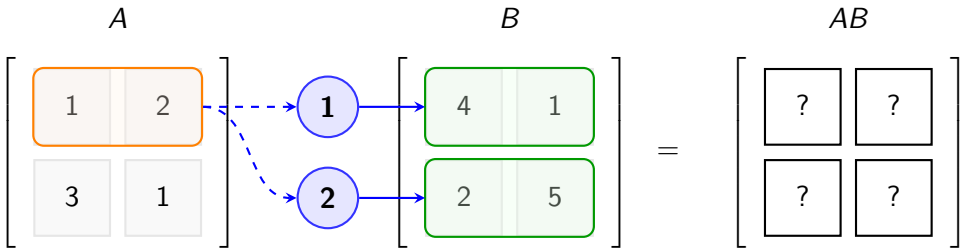

- Initial state
- Extract row $i$ from $A$

# Matrix Multiplication: Row Picture




- Initial state
- Extract row $i$ from $A$
- Rotate as coefficients (Weights), **acting on every row of** $B$

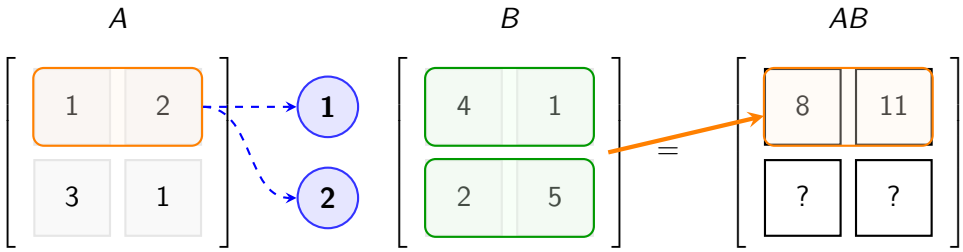
# Matrix Multiplication: Row Picture




$$1 \cdot [4, 1] + 2 \cdot [2, 5] = [8, 11]$$

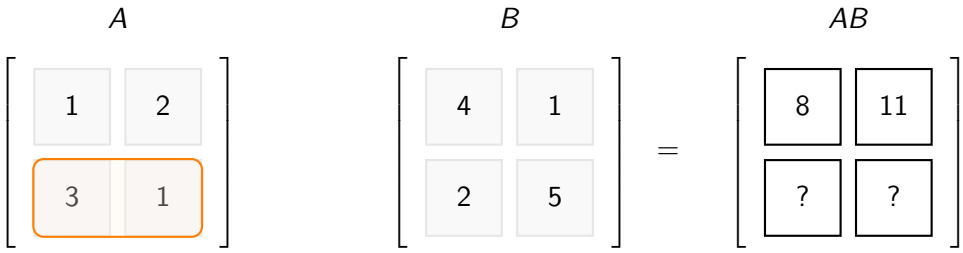
- Initial state
- Extract row $i$ from $A$
- Rotate as coefficients (Weights), **acting on every row of** $B$
- Linear combination of rows of $B$

# Matrix Multiplication: Row Picture



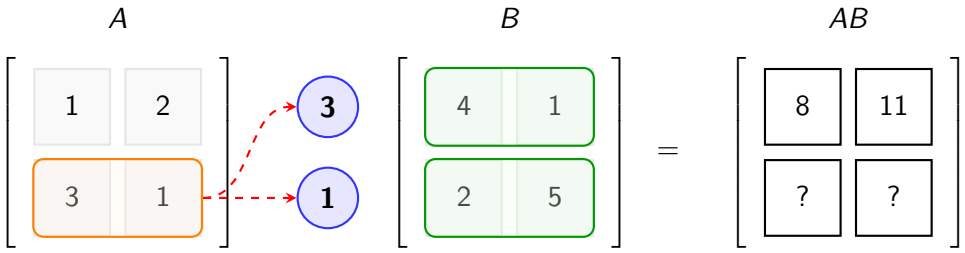

- Initial state
- Extract row $i$ from $A$
- Rotate as coefficients (Weights), **acting on every row of** $B$
- Linear combination of rows of $B$
- Fill into row $i$ of $AB$

# Matrix Multiplication: Row Picture



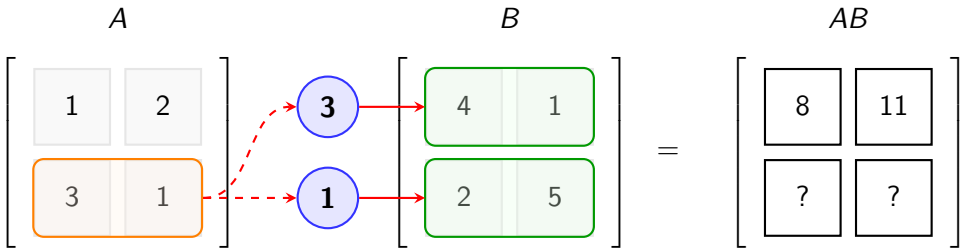

- Initial state
- Extract row $i$ from $A$
- Rotate as coefficients (Weights), **acting on every row of** $B$
- Linear combination of rows of $B$
- Fill into row $i$ of $AB$

# Matrix Multiplication: Row Picture




- Initial state
- Extract row $i$ from $A$
- Rotate as coefficients (Weights), **acting on every row of** $B$
- Linear combination of rows of $B$
- Fill into row $i$ of $AB$

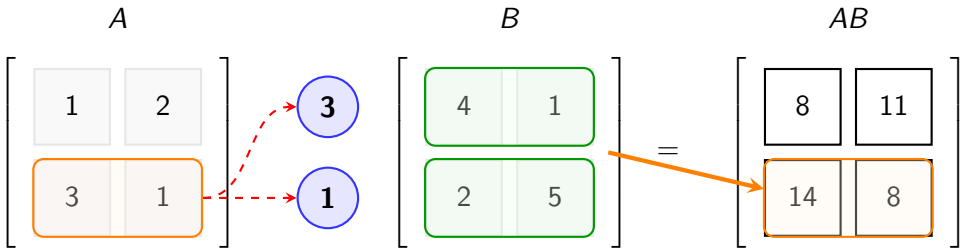
# Matrix Multiplication: Row Picture




$$3 \cdot [4, 1] + 1 \cdot [2, 5] = [14, 8]$$

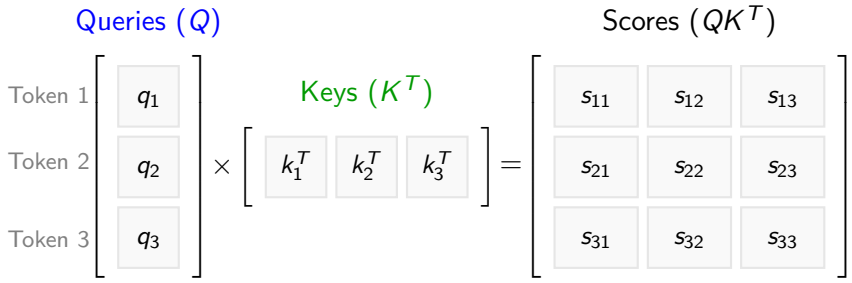
- Initial state
- Extract row $i$ from $A$
- Rotate as coefficients (Weights), **acting on every row of** $B$
- Linear combination of rows of $B$
- Fill into row $i$ of $AB$

# Matrix Multiplication: Row Picture



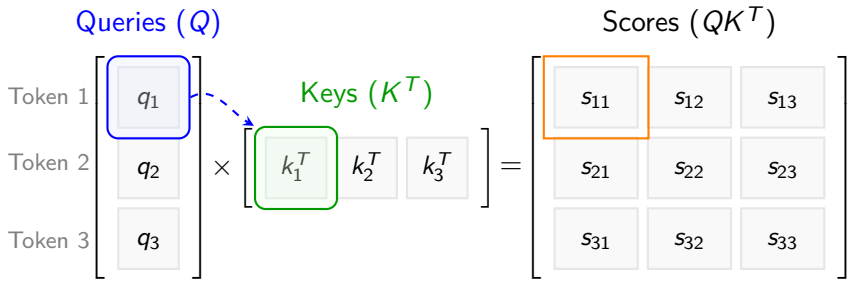

- Initial state
- Extract row $i$ from $A$
- Rotate as coefficients (Weights), **acting on every row of** $B$
- Linear combination of rows of $B$
- Fill into row $i$ of $AB$

# Attention Construction: Alignment & Masking




- **Relevance:** Dot product $q_i \cdot k_j$ measures similarity.

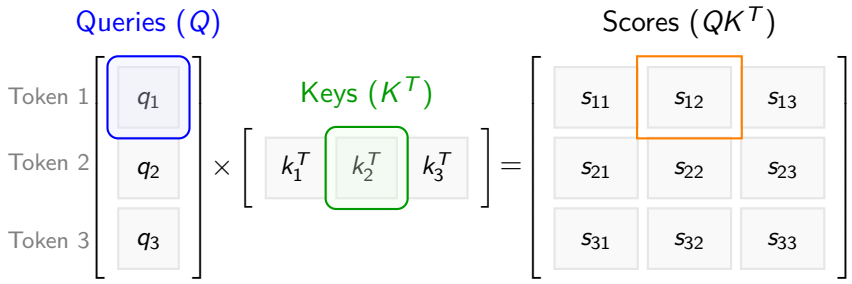
# Attention Construction: Alignment & Masking




$s_{11} = q_1 \cdot k_1$: "Self-attention (Token 1 $\leftrightarrow$ 1)"

- **Relevance:** Dot product $q_i \cdot k_j$ measures similarity.

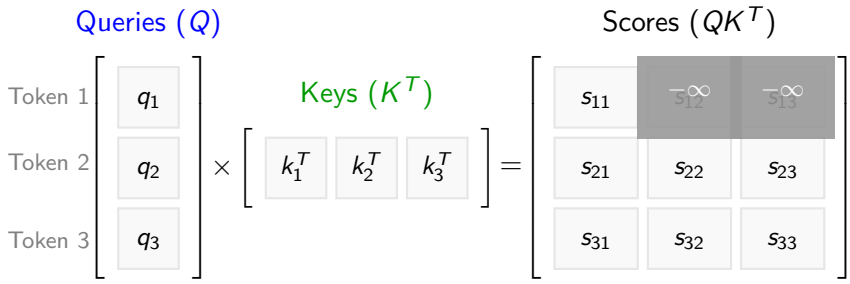
# Attention Construction: Alignment & Masking




$s_{12} = q_1 \cdot k_2$: "Token 1 looking at Token 2 (Future)..."

- **Relevance:** Dot product $q_i \cdot k_j$ measures similarity.
- **Masking:** Setting score to $-\infty$ prevents attending to future tokens.

# Attention Construction: Alignment & Masking

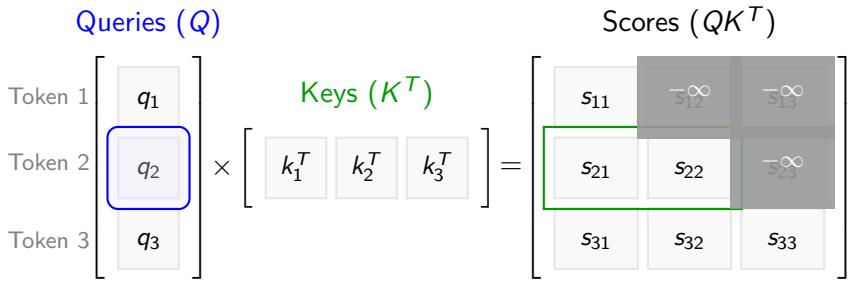



**CAUSAL MASK APPLIED!**

Score $\rightarrow$ $-\infty$ (Future is invisible).

- **Relevance:** Dot product $q_i \cdot k_j$ measures similarity.
- **Masking:** Setting score to $-\infty$ prevents attending to future tokens.

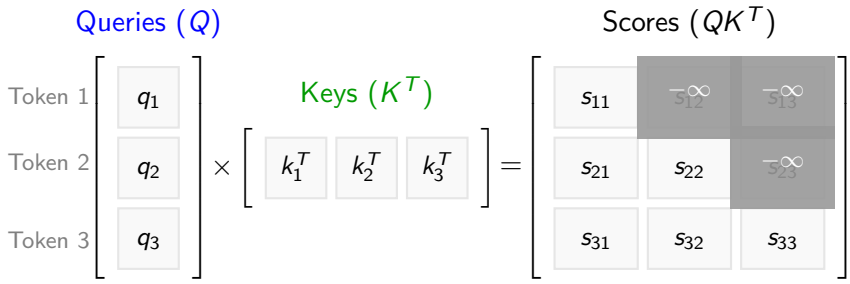
# Attention Construction: Alignment & Masking




Token 2 sees Past ($k_1$) & Present ($k_2$), but not Future ($k_3$).

- **Relevance:** Dot product $q_i \cdot k_j$ measures similarity.
- **Masking:** Setting score to $-\infty$ prevents attending to future tokens.

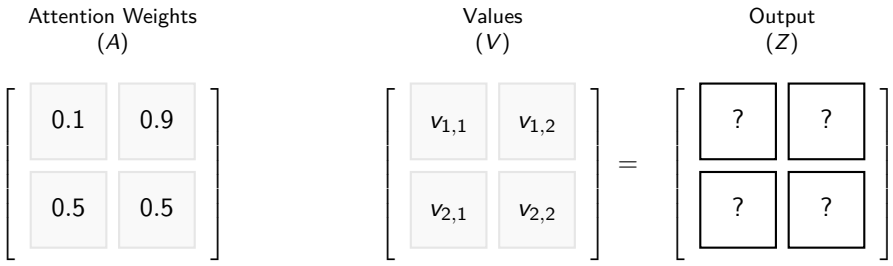
# Attention Construction: Alignment & Masking




The Score Matrix is now ready for Softmax.

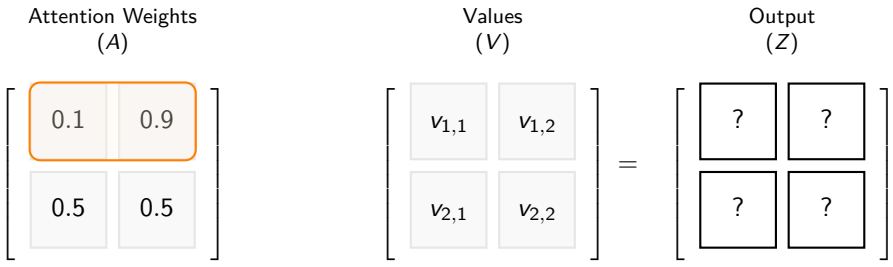
- **Relevance:** Dot product $q_i \cdot k_j$ measures similarity.
- **Masking:** Setting score to $-\infty$ prevents attending to future tokens.

# Attention Mechanism: Linear Combination of Values



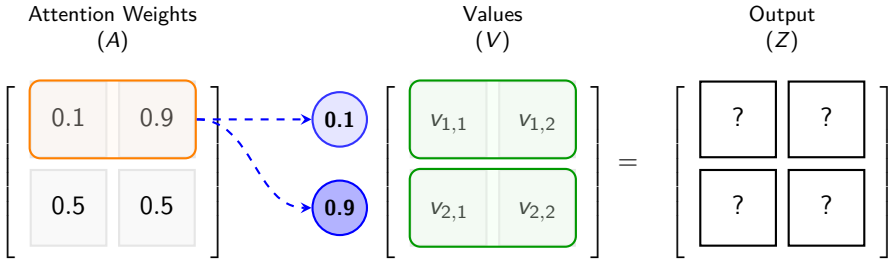

- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).

# Attention Mechanism: Linear Combination of Values



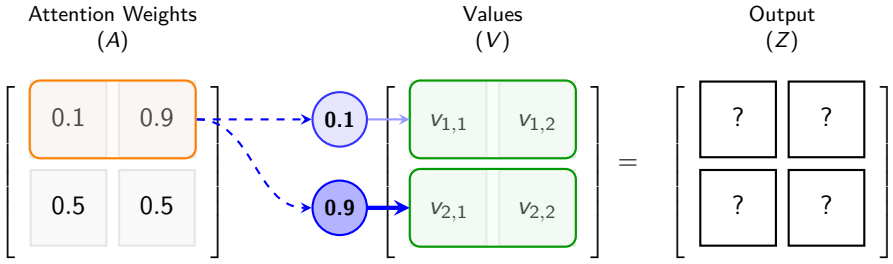

- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).
- **Coefficients:** The weights come from the Attention Matrix ($A$).

# Attention Mechanism: Linear Combination of Values




- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).
- **Coefficients:** The weights come from the Attention Matrix ($A$).
- **Fusion:** The output ($Z$) is a fused representation of relevant information.

# Attention Mechanism: Linear Combination of Values

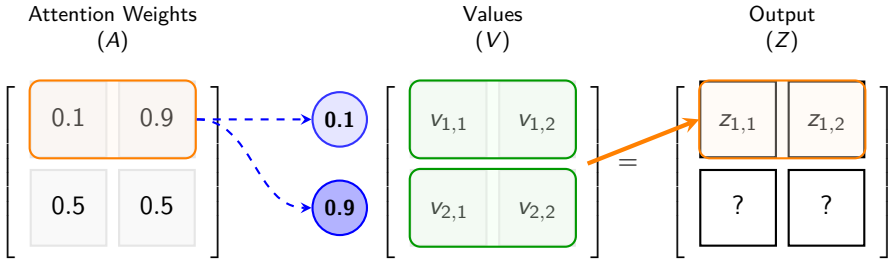



$$\mathbf{z}_1 = 0.1 \cdot \mathbf{v}_1 + \mathbf{0.9} \cdot \mathbf{v}_2$$

*(Linear combination dominated by $\mathbf{v}_2$)*

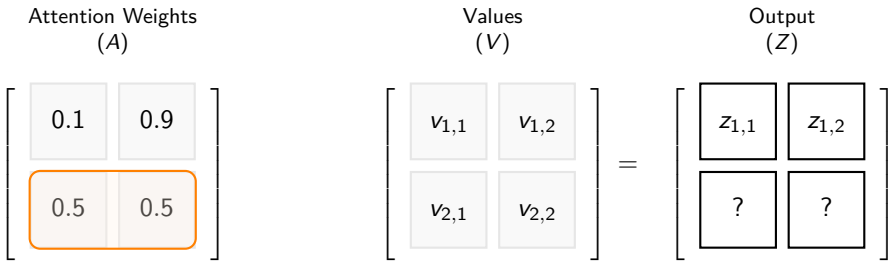
- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).
- **Coefficients:** The weights come from the Attention Matrix ($A$).
- **Fusion:** The output ($Z$) is a fused representation of relevant information.

# Attention Mechanism: Linear Combination of Values



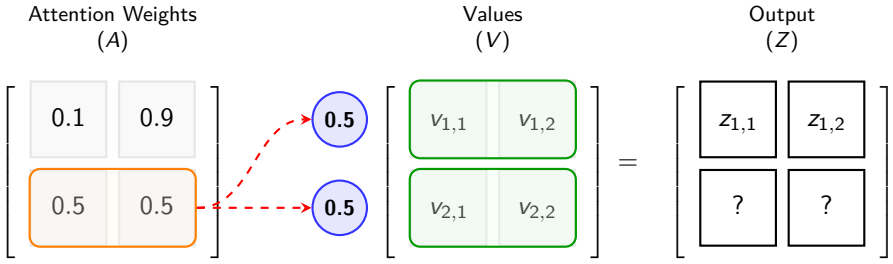

- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).
- **Coefficients:** The weights come from the Attention Matrix ($A$).
- **Fusion:** The output ($Z$) is a fused representation of relevant information.

# Attention Mechanism: Linear Combination of Values



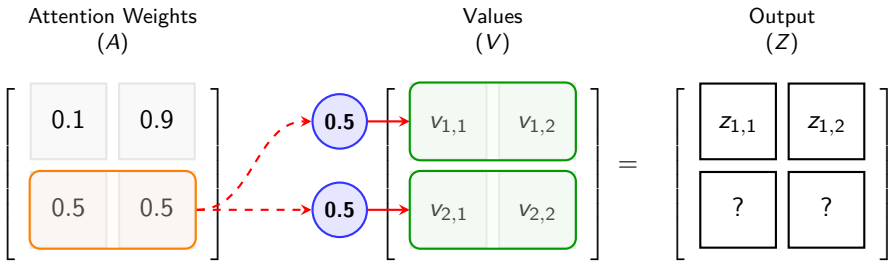

- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).
- **Coefficients:** The weights come from the Attention Matrix ($A$).
- **Fusion:** The output ($Z$) is a fused representation of relevant information.

# Attention Mechanism: Linear Combination of Values




- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).
- **Coefficients:** The weights come from the Attention Matrix ($A$).
- **Fusion:** The output ($Z$) is a fused representation of relevant information.

# Attention Mechanism: Linear Combination of Values

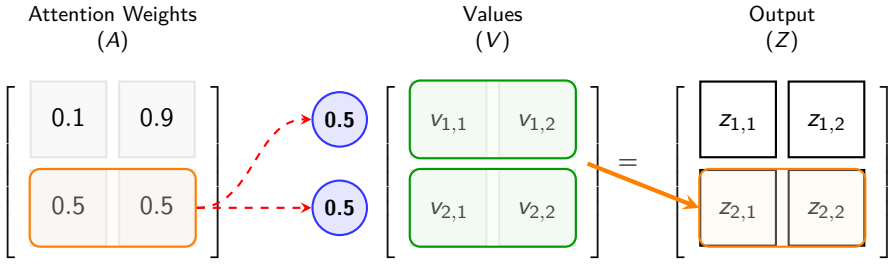



$$\mathbf{z}_2 = 0.5 \cdot \mathbf{v}_1 + 0.5 \cdot \mathbf{v}_2$$

*(Balanced mix of information)*

- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).
- **Coefficients:** The weights come from the Attention Matrix ($A$).
- **Fusion:** The output ($Z$) is a fused representation of relevant information.

# Attention Mechanism: Linear Combination of Values




- **Concept:** Attention is a **Linear Combination** of value vectors ($V$).
- **Coefficients:** The weights come from the Attention Matrix ($A$).
- **Fusion:** The output ($Z$) is a fused representation of relevant information.

# Self-Attention as Row-wise Linear Combination

Consider a single-head self-attention layer with sequence length $n$:

$$Q, K \in \mathbb{R}^{n \times d}, \qquad V \in \mathbb{R}^{n \times d_v}.$$

Define the attention scores and weights:

$$S = \frac{QK^\top}{\sqrt{d}} \in \mathbb{R}^{n \times n}, \qquad A = \mathrm{softmax}_{\mathrm{row}}(S) \in \mathbb{R}^{n \times n},$$

where softmax is applied row-wise.
The output of attention is

$$H = AV \in \mathbb{R}^{n \times d_v}.$$

Using the previous linear algebra fact, for each token index $i$:

$$H_{i*} = \sum_{j=1}^{n} A_{ij}\, V_{j*}.$$

**Interpretation:**

The representation of token $i$ is a linear combination of all value vectors $V_{j*}$, with weights $A_{ij}$.

# Attention as a Kernel-Weighted Sum

Define a (parametric) kernel

$$k(q_i, k_j) := \exp\Big(\frac{\langle q_i, k_j \rangle}{\sqrt{d}}\Big),$$

where $q_i$ and $k_j$ are the $i$-th and $j$-th rows of $Q$ and $K$.
Then the attention weights can be written as

$$A_{ij} = \frac{k(q_i, k_j)}{\sum_{\ell=1}^{n} k(q_i, k_\ell)}.$$

The output row becomes

$$H_{i*} = \sum_{j=1}^{n} \frac{k(q_i, k_j)}{\sum_{\ell=1}^{n} k(q_i, k_\ell)} V_{j*}.$$

**Kernel view of attention:**

- Values $\{V_{j*}\}$ are samples of a function on discrete points $\{x_j\}$.
- The kernel $k(q_i, k_j)$ measures the relevance between query $q_i$ and key $k_j$.
- The output $H_{i*}$ is a kernel-weighted average (normalized by the sum of kernel weights).

Thus self-attention is a *learnable, data-dependent kernel smoother* over the sequence.

# From Discrete Sum to Integral Operator

Think of the keys as points $\{x_j\}$ in a space $\Omega$ with a discrete measure

$$\mu = \sum_{j=1}^{n} \delta_{x_j},$$

and let $V(x_j) = V_{j*}$.
Then the attention output at query $q_i$ can be written as

$$H(q_i) = \frac{\sum_j k(q_i, x_j) V(x_j)}{\sum_j k(q_i, x_j)} = \frac{\int k(q_i, x) V(x) \, \mathrm{d}\mu(x)}{\int k(q_i, x) \, \mathrm{d}\mu(x)}.$$

In a continuum limit (dense sampling),

$$H(q) \approx \frac{\int_\Omega k_\theta(q, x) V(x) \, \mathrm{d}\mu(x)}{\int_\Omega k_\theta(q, x) \, \mathrm{d}\mu(x)}.$$

**Attention as a normalized kernel integral operator:**

$$\boxed{(\mathcal{T}_\theta V)(q) = \int_\Omega k_\theta(q, x) V(x) \, \mathrm{d}\mu(x) / \int_\Omega k_\theta(q, x) \, \mathrm{d}\mu(x).}$$

# Linear and Kernelized Attention as Operator Approximation

Suppose we approximate the kernel by a feature map

$$k_\theta(q, x) \approx \phi_\theta(q)^\top \phi_\theta(x),$$

with $\phi_\theta : \Omega \to \mathbb{R}^r$ (random or learned features).
In matrix form:

$$K(q_i, x_j) \approx \phi_\theta(q_i)^\top \phi_\theta(x_j), \quad \Phi_Q = \phi_\theta(Q) \in \mathbb{R}^{n \times r}, \quad \Phi_K = \phi_\theta(K) \in \mathbb{R}^{n \times r}.$$

Then

$$AV \approx \mathrm{row\_norm}(\Phi_Q \Phi_K^\top) V.$$

The unnormalized part factors as

$$\Phi_Q \Phi_K^\top V = \Phi_Q \, (\Phi_K^\top V).$$

**Operator-theoretic view:**

- Full self-attention is a dense kernel integral operator.
- Linear / kernelized attention uses a low-rank (feature-based) approximation of this operator.

# Connection to Nonlocal Operators and PDEs

Classical nonlocal operators often take the form

$$(\mathcal{L}u)(x) = \int_\Omega (u(y) - u(x)) K(x,y)\,\mathrm{d}y, \quad (\mathcal{N}u)(x) = \int_\Omega u(y)\,K(x,y)\,\mathrm{d}y.$$

Comparing with attention:

$$(\mathcal{T}_\theta V)(q) = \frac{\displaystyle\int_\Omega k_\theta(q,x) V(x)\,\mathrm{d}\mu(x)}{\displaystyle\int_\Omega k_\theta(q,x)\,\mathrm{d}\mu(x)}.$$

**Interpretation:**

- A self-attention layer is a *learnable nonlocal operator* acting on the value function $V$.
- Traditional PDE models prescribe $K(x,y)$ from physics.
- Transformers learn $k_\theta(q,x)$ from data in a high-dimensional latent space.

Stacking attention layers corresponds to composing multiple such nonlocal operators with pointwise nonlinearities, forming a deep, data-driven approximation of complex nonlocal dynamics.

# Summary

- **Linear algebra:** Left multiplication $C = AB$ makes each row $C_{i*}$ a linear combination of the rows of $B$.
- **Self-attention:** $H = AV$ means each token's output is a linear combination of all value vectors, with weights given by attention scores.
- **Kernel view:** Attention can be written as a normalized kernel-weighted sum and interpreted as a learnable kernel integral operator.
- **Approximation:** Linear and kernelized attention approximate this operator via low-rank or feature-based decompositions.
- **Nonlocal operators:** In a continuum perspective, self-attention is a learnable nonlocal operator, analogous to kernels in nonlocal PDEs and integral equations.

# Outline

# Motivation: Positional Information in Transformers

- Vanilla self-attention is permutation invariant:

$$\text{Attn}(X) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

- We need positional information to distinguish sequences like "A B C" vs. "C B A".
- Classical approaches:
  - *Absolute* positional encodings (e.g. sinusoidal or learned vectors added to token embeddings).
  - *Relative* positional encodings (e.g. bias as a function of $n - m$ added to attention logits).
- Absolute encodings tie the model to a fixed maximum length and do not naturally express relative positions.

# Idea of RoPE

- RoPE: encode position by applying a **rotation** to queries and keys in each attention head.
- For a token at position $m$, instead of

$$q_m = W_Q x_m, \quad k_m = W_K x_m,$$

RoPE uses

$$\tilde{q}_m = R_m q_m, \quad \tilde{k}_m = R_m k_m,$$

where $R_m$ is a block-diagonal rotation matrix depending on position $m$.

- Crucial property:

$$\langle \tilde{q}_m, \tilde{k}_n \rangle = \langle q_m, R_{n-m} k_n \rangle,$$

so the attention score depends on the **relative position** $n - m$.

# Rotations in 2D Subspaces

- Let head dimension be $d_h$, and group coordinates into 2D blocks:

$$q_m = (q_m^{(1)}, q_m^{(2)}, \ldots, q_m^{(d_h/2)}),$$

  where each $q_m^{(j)} \in \mathbb{R}^2$.

- For each block $j$ we pick a frequency $\omega_j > 0$ and define a 2D rotation:

$$R_m^{(j)} = \begin{bmatrix} \cos(\omega_j m) & -\sin(\omega_j m) \\ \sin(\omega_j m) & \cos(\omega_j m) \end{bmatrix}.$$

- The full rotation $R_m$ is block diagonal:

$$R_m = \mathrm{diag}(R_m^{(1)}, R_m^{(2)}, \ldots, R_m^{(d_h/2)}).$$

- Apply $R_m$ to $q_m$ and $k_m$ to get position-aware vectors $\tilde{q}_m, \tilde{k}_m$.

# Complex-Valued View and Relative Positions

- A 2D vector $q_m^{(j)} = (a, b)$ can be seen as a complex number $z = a + ib$.
- The rotation $R_m^{(j)}$ corresponds to multiplying by $e^{i\omega_j m}$:

$$\tilde{z}_m^{(j)} = e^{i\omega_j m} z_m^{(j)}.$$

- For positions $m$ and $n$:

$$\tilde{z}_m^{(j)} \overline{\tilde{z}_n^{(j)}} = e^{i\omega_j (m-n)} z_m^{(j)} \overline{z_n^{(j)}},$$

  so the phase difference depends only on $(m - n)$.
- Summing over all blocks yields

$$\langle \tilde{q}_m, \tilde{k}_n \rangle = \langle q_m, R_{n-m} k_n \rangle = f(q_m, k_n, n - m),$$

an attention score that is **equivariant to shifts** and carries relative positional information.

# Multi-Head Attention with RoPE

- For each head $h$:
$$q_m^{(h)} = W_Q^{(h)} x_m, \quad k_m^{(h)} = W_K^{(h)} x_m.$$
- Apply RoPE:
$$\tilde{q}_m^{(h)} = R_m^{(h)} q_m^{(h)}, \quad \tilde{k}_m^{(h)} = R_m^{(h)} k_m^{(h)}.$$
- Attention logits for head $h$:
$$s_{m,n}^{(h)} = \frac{1}{\sqrt{d_h}} \langle \tilde{q}_m^{(h)}, \tilde{k}_n^{(h)} \rangle.$$
- Frequencies $\{\omega_j\}$ are typically shared across heads and chosen as a geometric progression, analogous to sinusoidal positional encodings.

# Frequency Schedule and Length Extrapolation

- Frequencies are usually defined as

$$\omega_j = \theta^{-2j/d_h}, \quad \theta > 1,$$

  where $j = 0, 1, \ldots, d_h/2 - 1$.
- Low frequencies capture long-range structure, high frequencies capture fine-grained local patterns.
- Because RoPE is defined analytically for any integer position $m$, it can be *extrapolated* to longer sequences than seen during training (though with practical limits).
- Variants such as "scaled RoPE" or "XPOS" modify the effective frequency or add per-dimension scaling to improve length extrapolation and numerical stability.

# Outline

# Greedy decoding

The simplest generation strategy:

$$x_t = \arg\max_x p(x|\mathbf{x}_{t-1:1}; \boldsymbol{\theta}) \tag{18}$$

**Properties**:

- Deterministic: Same input always produces same output
- Fast: Single forward pass per token
- Often suboptimal: Local decisions may not be globally optimal

**Issues**:

- Repetition: May get stuck in loops
- Lack of diversity: Always chooses most likely token
- Mode collapse: May not explore the full distribution

# Random sampling

Sample from the full distribution:

$$x_t \sim p(x|\mathbf{x}_{t-1:1}; \boldsymbol{\theta}) \tag{19}$$

**Advantages**:

- Diversity: Explores the full distribution
- Natural: Reflects model uncertainty
- Flexibility: Can control randomness

**Challenges**:

- Quality control: May generate low-quality sequences
- Consistency: Less predictable output
- Evaluation: Harder to assess quality

# Temperature scaling and nucleus sampling

**Temperature scaling**:

$$p_{\text{temp}}(x|\mathbf{x}_{t-1:1}) = \frac{\exp(f(x, \mathbf{x}_{t-1:1})/\tau)}{\sum_{x'} \exp(f(x', \mathbf{x}_{t-1:1})/\tau)} \tag{20}$$

where $x$ is a token candidate from the vocabulary $\mathcal{V}$ for $x_t$, $f(x, x_{t-1:1})$ is the logit (pre-softmax output) for $x$ given the history $x_{t-1:1}$, and $\tau > 0$ controls randomness:

- $\tau \to 0$: Greedy decoding
- $\tau = 1$: Original distribution
- $\tau > 1$: More random

**Nucleus sampling, i.e., top-$p$ sampling** (Holtzman et al., 2019):

$$\text{Select smallest } V \text{ such that } \sum_{x \in V} p(x|\mathbf{x}_{t-1:1}) \geq p \tag{21}$$

Then sample from the truncated distribution.

# Limiting behavior of temperature scaling I

Consider logits $f(x, \mathbf{x}_{t-1:1})$ over a finite dictionary $\mathcal{V}$ and the temperature-scaled distribution

$$p_\tau(x) = \frac{\exp(f(x, \mathbf{x}_{t-1:1})/\tau)}{\sum_{x' \in \mathcal{V}} \exp(f(x', \mathbf{x}_{t-1:1})/\tau)}.$$

**Claim 1 (greedy limit).** Let $x^\star = \arg\max_x f(x, \mathbf{x}_{t-1:1})$ and assume it is unique. Then

$$\lim_{\tau \to 0^+} p_\tau(x) = \delta_{x^\star}(x).$$

**Proof.** For any $x \neq x^\star$, we have $f(x, \mathbf{x}_{t-1:1}) - f(x^\star, \mathbf{x}_{t-1:1}) < 0$, hence

$$\frac{p_\tau(x)}{p_\tau(x^\star)} = \exp\left(\frac{f(x, \mathbf{x}_{t-1:1}) - f(x^\star, \mathbf{x}_{t-1:1})}{\tau}\right) \xrightarrow[\tau \to 0^+]{} 0,$$

so all mass concentrates on $x^\star$. (We can also divided by $exp(f(x^\star, \mathbf{x}_{t-1:1})/\tau)$ on $p_\tau(x)$ and $p_\tau(x^\star)$ to get the same result with $p_\tau(x^\star) = 1$ and $p_\tau(x) = 0$ for $x \neq x^\star$.)

# Limiting behavior of temperature scaling II

We keep the same definition

$$p_\tau(x) = \frac{\exp(f(x, \mathbf{x}_{t-1:1})/\tau)}{\sum_{x' \in \mathcal{V}} \exp(f(x', \mathbf{x}_{t-1:1})/\tau)}.$$

**Claim 2 (uniform limit).** As $\tau \to \infty$,

$$p_\tau(x) \longrightarrow \frac{1}{|\mathcal{V}|} \quad \text{for all } x \in \mathcal{V}.$$

**Proof.** Using $\exp(f/\tau) = 1 + \mathcal{O}(1/\tau)$ as $\tau \to \infty$, we have

$$p_\tau(x) = \frac{1 + \mathcal{O}(1/\tau)}{|\mathcal{V}| + \mathcal{O}(1/\tau)} \xrightarrow[\tau \to \infty]{} \frac{1}{|\mathcal{V}|}.$$

# Outline

# Evaluation metrics for autoregressive models

**Perplexity**:

$$\text{Perplexity} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p(x_t|\mathbf{x}_{t-1:1};\boldsymbol{\theta})\right) \tag{22}$$

**Other metrics**:

- BLEU: N-gram overlap with reference
- ROUGE: Recall-oriented evaluation
- BERTScore: Semantic similarity using BERT
- Human evaluation: Subjective quality assessment

**Challenges**:

- No single metric: Different tasks need different metrics
- Reference dependence: Many metrics need reference text
- Human evaluation: Expensive and subjective

# Information-theoretic perspective

The entropy of an autoregressive model is:

$$H(p_\theta) = \mathbb{E}_{\mathbf{x}_{1:T} \sim p_\theta} \left[ -\sum_{t=1}^{T} \log p(x_t | \mathbf{x}_{t-1:1}; \theta) \right] \tag{23}$$

**Key insights**:

- Conditional entropy: $H(X_t|X_{t-1:1}) \leq H(X_t)$
- Chain rule: $H(X_{1:T}) = \sum_{t=1}^{T} H(X_t|X_{t-1:1})$
- Compression: Autoregressive models compress information

**Compression ratio**:

$$\text{Compression} = \frac{H(X_{1:T})}{T \log V} = \frac{\sum_{t=1}^{T} H(X_t|X_{t-1:1})}{T \log V} \tag{24}$$

Lower compression ratio means better modeling of dependencies.

# Large language models and scaling laws

**Scaling laws** (Kaplan et al., 2020):

$$L(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{25}$$

where $N$ is model size, $D$ is dataset size, and $L$ is loss.

**Key findings**:

- Power law scaling: Performance improves predictably with scale
- Emergent abilities: New capabilities emerge at certain scales
- Efficient scaling: Larger models are more sample-efficient

**Implications**:

- Compute requirements: Training costs scale dramatically
- Data requirements: Need massive datasets
- Architecture choices: Transformer scales well

# Common challenges and limitations

**Training challenges**:

1. Exposure bias: Training uses ground truth, inference uses model predictions
2. Teacher forcing mismatch: Different distributions during training and inference
3. Long sequences: Gradient vanishing/exploding

**Generation challenges**:

1. Repetition: Models may repeat phrases or sentences
2. Coherence: Long-range dependencies hard to maintain
3. Diversity vs. Quality: Trade-off between exploration and exploitation

**Computational challenges**:

1. Sequential generation: Cannot parallelize inference
2. Memory requirements: Need to store full history
3. Scalability: Training large models is expensive

# Outline



北京雁栖湖应用数学研究院
Beijing Institute of Mathematical Sciences and Applications

# Open questions

If you are interested in these questions, please feel free to write an email to me (hpp@bimsa.cn) with your comments.

1. How can we design better non-autoregressive models that maintain quality while enabling parallel generation?
2. Could we use auto-regressive models to model the joint distribution of images and text naturally?
3. How do we balance the trade-off between generation speed and quality in practical applications?
4. What role do autoregressive models play in the broader landscape of generative AI, and how do they compare to diffusion models and other approaches?
5. Can we develop theoretical frameworks to understand when autoregressive models will succeed or fail?

# Welcome inputs

Any comments?

Welcome your inputs!